

RAG Chatbot Design Document

Problem Statement

Build a simple, local-first chatbot for React documentation. The system should answer questions from a small set of local markdown files and return answers grounded in that documentation.

Architecture

This project uses a simple retrieval-augmented generation (RAG) architecture rather than a full agentic or ReAct-style workflow.

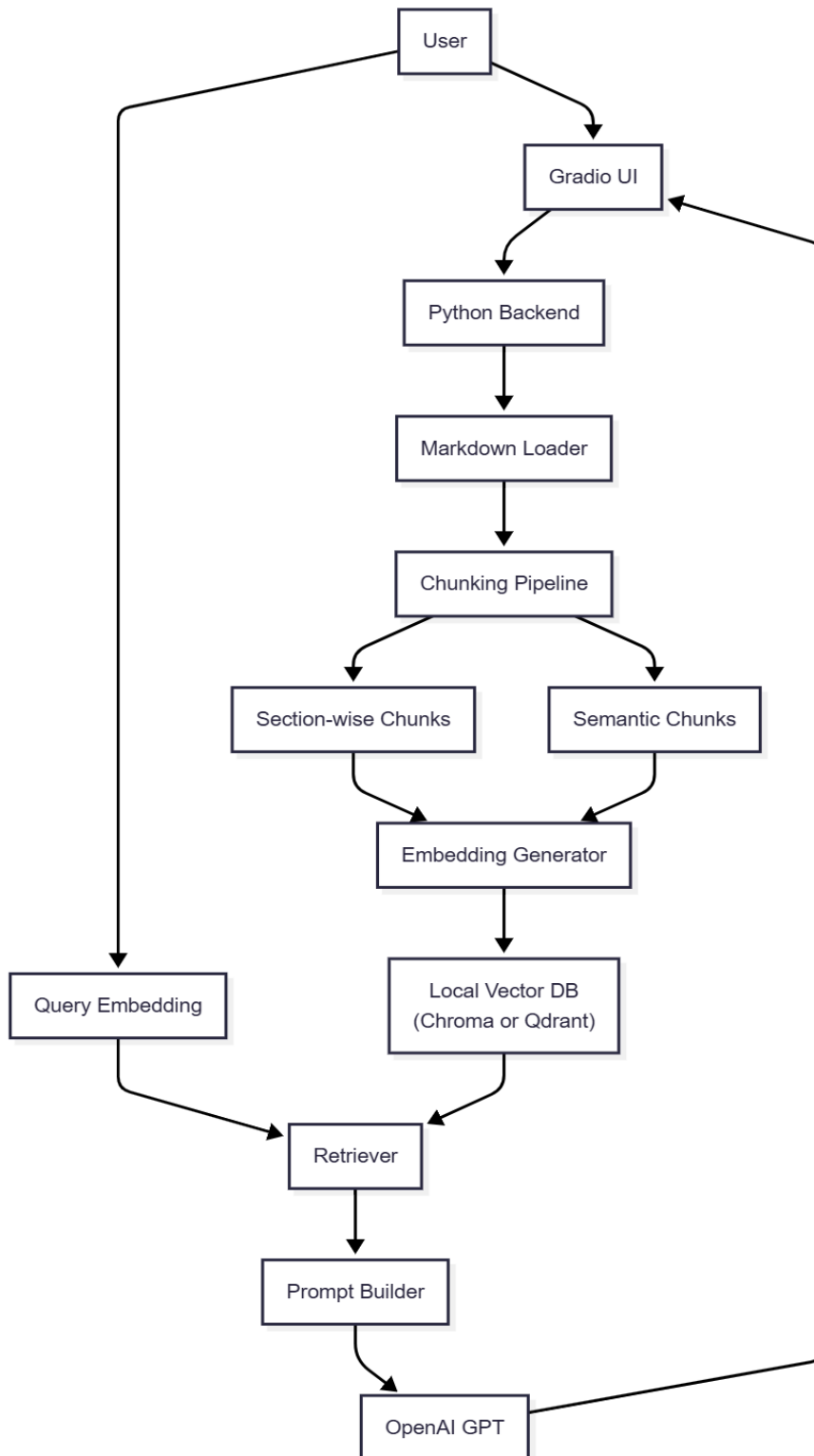
High-level flow

1. A user enters a question in the chat UI.
2. The backend embeds the user query.
3. The system retrieves the most relevant chunks from a local vector database.
4. The retrieved context is passed to OpenAI along with the user question.
5. OpenAI generates a final answer grounded in the retrieved content.
6. The answer is returned to the user in the UI.

Components

- UI: Gradio for the MVP
- Backend: Python
- Knowledge source: local markdown files
- Chunking: two approaches
 - section-wise chunking
 - semantic chunking
- Embeddings: sentence-transformers
- Vector database: Chroma or Qdrant (local-first)
- LLM: OpenAI API for final answer generation only

Design diagram



Why this architecture

- It is easy to understand and implement
- It keeps retrieval and generation separate
- It is suitable for a static local knowledge base
- It avoids unnecessary complexity

Note on ReAct

This version does not use ReAct or a multi-step reasoning loop. The system is intentionally simple: retrieve relevant context, then generate an answer from that context.

Evaluation Criteria

1. Create 20–30 sample questions from the React documentation markdown files.
2. Run each question through the RAG pipeline and record the retrieved chunks and the generated answer.
3. Check whether the retrieved chunks match the question topic.
4. Check whether the answer is grounded in the retrieved context and does not introduce unsupported claims.
5. Check whether the answer directly addresses the question.

Rule-based evals

- Retrieval relevance: verify that the retrieved chunks overlap with the main terms or concepts in the question.
- Answer groundedness: verify that the final answer is supported by the retrieved context.
- Answer relevance: verify that the answer directly addresses the question instead of giving generic or unrelated content.

Framework Justification

This project does not use orchestration frameworks such as Pydantic AI, smolagents, or LangGraph. The MVP uses a lightweight Python stack instead:

- Gradio for UI
- Python for backend logic
- sentence-transformers for embeddings
- Chroma or Qdrant for vector storage
- OpenAI for final answer generation